

Word2Vec_FastText_tutorial

July 26, 2026

1 Word2Vec and FastText Tutorial

1.1 Ye Kyaw Thu, Lab Leader, Language Understanding Lab, Myanmar

1.2 Date: 26 July 2026

မနေ့က လက်ချာမှာလည်း သင်ပြပေးခဲ့သလို Word2Vec ဆိုတဲ့ ပရိုပိုဇယ်ကနေ အဓိပ္ပါယ်ဆင်တူတဲ့ စာလုံးတွေကို ဆွဲထုတ် လို့ရလာပါတယ်။ အဲဒီ ကနေ word embedding အပိုင်းမှာလည်း အများကြီး အဆင်ပြေလာပါတယ်။ FastText ကတော့ Word2Vec ကို အခြေခံပြီးနောက်ပိုင်းမှ ထွက်လာတဲ့ ပိုကောင်းတဲ့ ပရိုပိုဇယ်ပါ။

1.3 Word2Vec

Core Idea: Represents words as continuous vector spaces where words sharing common contexts are mapped close to each other. It uses two main architectures: CBOW (Continuous Bag-of-Words, predicting a word from its context) and Skip-gram (predicting context words from a target word).

Original Paper: [Efficient Estimation of Word Representations in Vector Space](#) (Mikolov et al., 2013)

One more paper: <https://aclanthology.org/N13-1090.pdf>

Word2Vec Wiki: <https://en.wikipedia.org/wiki/Word2vec>

Google Code Archive: **Original C Implementation:** <https://code.google.com/archive/p/word2vec/>
Gensim Library for Word2Vec and FastText: <https://pypi.org/project/gensim/>

```
[27]: %pwd
```

```
[27]: '/home/ye/aif2/word-seg'
```

```
[2]: !!pip install gensim --break-system-packages
```

```
[2]: ['Defaulting to user installation because normal site-packages is not
writeable',
'Requirement already satisfied: gensim in /home/ye/.local/lib/python3.12/site-
packages (4.4.0)',
'Requirement already satisfied: numpy>=1.18.5 in
/home/ye/.local/lib/python3.12/site-packages (from gensim) (2.1.3)',
'Requirement already satisfied: scipy>=1.7.0 in
/home/ye/.local/lib/python3.12/site-packages (from gensim) (1.16.0)',
'Requirement already satisfied: smart_open>=1.8.1 in
/home/ye/.local/lib/python3.12/site-packages (from gensim) (7.5.0)',
```

```
'Requirement already satisfied: wrapt in /home/ye/.local/lib/python3.12/site-packages (from smart_open>=1.8.1->gensim) (1.17.2)']
```

1.4 Import Libraries and Verify Corpus Path

```
[28]: import os
from gensim.models import Word2Vec
from gensim.models.fasttext import FastText

# Define your big corpus path
corpus_path = "/home/ye/aif2/word-seg/sub-word/big_data/big_corpus.txt"

# Verify file exists and check line count
if os.path.exists(corpus_path):
    with open(corpus_path, "r", encoding="utf-8") as f:
        line_count = sum(1 for _ in f)
        print(f"Corpus loaded successfully! Total lines: {line_count}")
else:
    print("Error: Corpus path not found. Please check path.")
```

Corpus loaded successfully! Total lines: 43196

1.5 Prepare a Streaming Sentence Generator

Corpus ဖိုင်တစ်ခုလုံးကို RAM ပေါ်ကိုတခါတည်း ဆွဲတင်တာမျိုး မလုပ်ပါနဲ့။ Gensim မှာ စာကြောင်းတွေကို တစ်ကြောင်းချင်းစီဖတ်ပြီး loop ပတ်သွားလို့ ရပါတယ်။ ဆရာတို့ရဲ့ corpus ထဲမှာက word ဖြစ်ထားပြီးသားမို့ space နဲ့ မြန်မာစာလုံးတွေကို ဖြတ်ယူလိုက်ပြီး အဲဒီကနေ word2vec ဆောက်သွားလို့ ရပါတယ်။

```
[29]: class MyanmarCorpusStream:
    """A memory-friendly stream reader for line-by-line tokenized text files."""
    def __init__(self, filepath):
        self.filepath = filepath

    def __iter__(self):
        with open(self.filepath, "r", encoding="utf-8") as f:
            for line in f:
                tokens = line.strip().split()
                if tokens: # Yield non-empty lines
                    yield tokens

# Instantiate the stream
sentences = MyanmarCorpusStream(corpus_path)
```

1.6 Train Word2Vec (Skip-gram Model)

အောက်ပါ setting နဲ့ word2vec မော်ဒယ် ဆောက်ကြည့်မယ်။

- vector_size=100 dimensions

- a context window=5
- min_count=2 times
- sg=1 (which specifies Skip-gram architecture)

```
[30]: print("Training Word2Vec (Skip-gram) on Myanmar corpus...")
w2v_model = Word2Vec(
    sentences=sentences,
    vector_size=100,      # Dimensionality of word vectors
    window=5,            # Context window size
    min_count=2,          # Ignore rare words (မြန်မာစာလိုမျိုး corpus size က
    # လေးတဲ့အခါမှာတော့ min_count တန်ဖိုးကို တခါတလေ ၂ လောက် ထားတာမျိုးလည်း
    # လုပ်ရပါတယ်)
    workers=16,           # Parallel worker threads (adjust based on your RTX
    # 3090 Ti CPU cores)
    sg=1                  # 1 for Skip-gram, 0 for CBOW
)

print(f"Word2Vec vocabulary size: {len(w2v_model.wv)}")

# Save model for future use
os.makedirs("models", exist_ok=True)
w2v_model.save("models/myanmar_word2vec.model")
print("Word2Vec model saved successfully.")
```

Training Word2Vec (Skip-gram) on Myanmar corpus...
Word2Vec vocabulary size: 11260
Word2Vec model saved successfully.

1.7 Train FastText Model

```
[31]: print("Training FastText on Myanmar corpus...")
ft_model = FastText(
    sentences=sentences,
    vector_size=100,
    window=5,
    min_count=2,
    workers=4,
    sg=1,                  # Skip-gram architecture
    min_n=3,              # Min char ngram length
    max_n=6               # Max char ngram length
)

print(f"FastText vocabulary size: {len(ft_model.wv)}")

# Save model
ft_model.save("models/myanmar_fasttext.model")
print("FastText model saved successfully.")
```

```
Training FastText on Myanmar corpus...
FastText vocabulary size: 11260
FastText model saved successfully.
```

1.8 Word Similarity Measurements

ခုဆိုရင် ဆရာတို့မှာ Word2Vec မော်ဒယ်ရော၊ FastText မော်ဒယ်ရော ရှိပြီ။ မြန်မာစာ စာလုံး တချို့ကို ရိုက်ထည့်ပြီး similarity အဖြစ်ဆုံး စာလုံးတွေကို ဆွဲထုတ်ပြီး ရလဒ်တွေကို လေ့လာကြည့်ကြရအောင်။

```
[32]: # Choose a target word present in your dataset
target_word = "ကျောင်းသား"

print(f"--- Top Similar Words to '{target_word}' using Word2Vec ---")
try:
    for word, sim in w2v_model.wv.most_similar(target_word, topn=5):
        print(f" {word}: {sim:.4f}")
except KeyError:
    print(f"'{target_word}' not found in Word2Vec vocabulary.")

print(f"\n--- Top Similar Words to '{target_word}' using FastText ---")
try:
    for word, sim in ft_model.wv.most_similar(target_word, topn=5):
        print(f" {word}: {sim:.4f}")
except KeyError:
    print(f"'{target_word}' not found in FastText vocabulary.")
```

```
--- Top Similar Words to 'ကျောင်းသား' using Word2Vec ---
```

```
ကျောင်းသူ: 0.8185
လူငယ်: 0.7911
ပညာသင်: 0.7903
မိတ်ဆွေ: 0.7805
စာရေးဆရာ: 0.7737
```

```
--- Top Similar Words to 'ကျောင်းသား' using FastText ---
```

```
ကျောင်းသားကံဒ်: 0.9833
ကျောင်းသားရေးရာ: 0.9777
ကျောင်းသူ: 0.9728
နွားကျောင်းသား: 0.9619
ကျောင်းဆရာ: 0.9564
```

```
[33]: # Choose a target word present in your dataset
target_word = "အဖေ"

print(f"--- Top Similar Words to '{target_word}' using Word2Vec ---")
try:
    for word, sim in w2v_model.wv.most_similar(target_word, topn=5):
```

```

        print(f" {word}: {sim:.4f}")
except KeyError:
    print(f"'{target_word}' not found in Word2Vec vocabulary.")

print(f"\n--- Top Similar Words to '{target_word}' using FastText ---")
try:
    for word, sim in ft_model.wv.most_similar(target_word, topn=5):
        print(f" {word}: {sim:.4f}")
except KeyError:
    print(f"'{target_word}' not found in FastText vocabulary.")

```

--- Top Similar Words to 'အဖေ' using Word2Vec ---

```

ညီမလေး: 0.9300
ကောင်လေး: 0.9269
အမေ: 0.9247
ချစ်သူ: 0.9190
ကောင်မလေး: 0.9166

```

--- Top Similar Words to 'အဖေ' using FastText ---

```

အဖေ: 0.9330
အမေ: 0.9187
ညီမလေး: 0.9025
အမေ့: 0.8975
သူငွေ: 0.8972

```

[34]: *# Choose a target word present in your dataset*

```

target_word = "ချစ်"

print(f"--- Top Similar Words to '{target_word}' using Word2Vec ---")
try:
    for word, sim in w2v_model.wv.most_similar(target_word, topn=5):
        print(f" {word}: {sim:.4f}")
except KeyError:
    print(f"'{target_word}' not found in Word2Vec vocabulary.")

print(f"\n--- Top Similar Words to '{target_word}' using FastText ---")
try:
    for word, sim in ft_model.wv.most_similar(target_word, topn=5):
        print(f" {word}: {sim:.4f}")
except KeyError:
    print(f"'{target_word}' not found in FastText vocabulary.")

```

--- Top Similar Words to 'ချစ်' using Word2Vec ---

```

အံ့ဩ: 0.9185
သဘောကျ: 0.9127
သနား: 0.9126

```

သဘောကောင်း: 0.9122
အားကျ: 0.9046

--- Top Similar Words to 'ချစ်' using FastText ---

ချစ်သူ: 0.9504
အတွေး: 0.9314
ချစ်စရာ: 0.9269
စိတ်ညစ်: 0.9227
ချွေးမ: 0.9136

```
[35]: # Choose a target word present in your dataset
target_word = "ဂျပန်"

print(f"--- Top Similar Words to '{target_word}' using Word2Vec ---")
try:
    for word, sim in w2v_model.wv.most_similar(target_word, topn=5):
        print(f" {word}: {sim:.4f}")
except KeyError:
    print(f"'{target_word}' not found in Word2Vec vocabulary.")

print(f"\n--- Top Similar Words to '{target_word}' using FastText ---")
try:
    for word, sim in ft_model.wv.most_similar(target_word, topn=5):
        print(f" {word}: {sim:.4f}")
except KeyError:
    print(f"'{target_word}' not found in FastText vocabulary.")
```

--- Top Similar Words to 'ဂျပန်' using Word2Vec ---

ပြင်သစ်: 0.8263
သံရုံး: 0.8049
အင်္ဂလန်: 0.7851
သံတမန်: 0.7759
ဗြိတိသျှ: 0.7698

--- Top Similar Words to 'ဂျပန်' using FastText ---

မြန်မာပြည်: 0.8551
အင်္ဂလန်: 0.8426
ဗြိတိန်: 0.8373
မြန်မာနိုင်ငံ: 0.8361
ဂျာမနီ: 0.8360

```
[36]: # Choose a target word present in your dataset
target_word = "ပညာ"

print(f"--- Top Similar Words to '{target_word}' using Word2Vec ---")
```

```

try:
    for word, sim in w2v_model.wv.most_similar(target_word, topn=5):
        print(f" {word}: {sim:.4f}")
except KeyError:
    print(f"'{target_word}' not found in Word2Vec vocabulary.")

print(f"\n--- Top Similar Words to '{target_word}' using FastText ---")
try:
    for word, sim in ft_model.wv.most_similar(target_word, topn=5):
        print(f" {word}: {sim:.4f}")
except KeyError:
    print(f"'{target_word}' not found in FastText vocabulary.")

```

--- Top Similar Words to 'ပညာ' using Word2Vec ---

သိပ္ပံ: 0.8990
 ပညာရပ်: 0.8637
 ဘာသာရပ်: 0.8276
 ဝိဇ္ဇာ: 0.8210
 ဘောဂ: 0.8065

--- Top Similar Words to 'ပညာ' using FastText ---

သိပ္ပံပညာ: 0.9058
 ပညာရပ်: 0.8947
 သိပ္ပံ: 0.8864
 သိပ္ပံပညာရှင်: 0.8791
 ပညာရှိ: 0.8707

1.9 Handling Out-Of-Vocabulary (OOV) Words

FastText က OOV ကို handle လုပ်ပေးနိုင်ပါတယ်။

[37]: `oov_word = "လေးဖြူ" # A diminutive or modified form that might be rare/missing`

```

print("Testing Word2Vec with OOV word:")
try:
    print(w2v_model.wv[oov_word])
except KeyError as e:
    print(f"Caught Expected Error in Word2Vec: {e} (Word2Vec cannot generate_
    ↪ vectors for unseen words).")

print("\nTesting FastText with OOV word:")
try:
    # FastText generates a vector dynamically using subword n-grams
    vector = ft_model.wv[oov_word]
    print(f"Success! Generated vector shape for OOV word '{oov_word}': {vector.
    ↪ shape}")

```

```

# Find similar words to this unseen OOV word
print("Most similar words to OOV term using FastText subwords:")
for word, sim in ft_model.wv.most_similar(oov_word, topn=3):
    print(f" {word}: {sim:.4f}")
except Exception as e:
    print(f"Error: {e}")

```

Testing Word2Vec with OOV word:

Caught Expected Error in Word2Vec: "Key 'လေးဖြူ' not present" (Word2Vec cannot generate vectors for unseen words).

Testing FastText with OOV word:

Success! Generated vector shape for OOV word 'လေးဖြူ': (100,)

Most similar words to OOV term using FastText subwords:

```

လေးလံ: 0.9689
ပါးနပ်: 0.9603
ဖားငယ်မ: 0.9598

```

1.10 Analogy Algebra Calculation ($A - B + C = ?$)

Word2Vec က တကယ်က စာလုံးတွေရဲ့ အဓိပ္ပါယ်ကို လူတွေ နားလည်သလို နားလည်တာ မဟုတ်ပါဘူး။ သူ့ အလုပ်လုပ်ပုံက စာလုံးတွေကို high-dimensional space ရဲ့ coordinate အနေနဲ့ ကြည့်တာပါ။ ဥပမာ 100 dimension နဲ့ ဆောက်ထားရင် 100 dimensional space ဖြစ်ပြီး 300 dimension နဲ့ မော်ဒယ်ဆောက်ထားရင် 300 dimensional space ပါ။ အဲဒီ မှာ အဓိပ္ပါယ်နီးစပ်တဲ့စကားလုံးတွေ ဒါမှမဟုတ် ဆက်စပ်မှုရှိတဲ့ စကားလုံးတွေက အုပ်စုလိုမျိုး ဖြစ်နေကြပါတယ်။ အဲဒါကြောင့် vector arithmetic နဲ့ အပေါင်း၊ အနှုတ် လုပ်ပြီး အဓိပ္ပါယ်ဆင်တူတဲ့ စာလုံးတွေကို ဆွဲထုတ်လို့ ရနိုင်ပါတယ်။

ဒါပေမဲ့ စာလုံးသေသေချာချာဖြစ်တောက်ထားတဲ့ corpus အကြီးဖြစ်မှ ရလဒ်က ကောင်းတာပါ။

လက်ရှိ စာကြောင်းရေ လေးသောင်းကျော်နဲ့ ဆောက်ထားတဲ့ Word2Vec, FastText မော်ဒယ်ကိုပဲ သုံးပြီး analogy algebra calculation လုပ်ကြည့်ပြီး လေ့လာကြည့်ကြရအောင်။

```

[38]: def perform_analogy(model, positive_words, negative_words):
    print(f"Analogy: Positive {positive_words} - Negative {negative_words}")
    try:
        results = model.wv.most_similar(positive=positive_words,
        ↪negative=negative_words, topn=3)
        for word, sim in results:
            print(f" -> {word} (score: {sim:.4f})")
    except KeyError as e:
        print(f"Skipped due to missing vocabulary item: {e}")

# Example analogy framework (ensure these terms exist in your big_corpus.txt
↪vocabulary)
# Format: model.wv.most_similar(positive=['word_b', 'word_c'],
↪negative=['word_a']) -> corresponds to word_a is to word_b as word_c is to ?

```



```

print("--- Word2Vec Analogy Test ---")
# Adjust these tokens based on common entities present in your corpus text
perform_analogy(w2v_model, positive_words=["နေပြည်တော်", "မြန်မာ"],
    ↪negative_words=["ထိုင်း"])

print("\n--- FastText Analogy Test ---")
perform_analogy(ft_model, positive_words=["နေပြည်တော်", "မြန်မာ"],
    ↪negative_words=["ထိုင်း"])

```

```

--- Word2Vec Analogy Test ---
Analogy: Positive ['နေပြည်တော်', 'မြန်မာ'] - Negative ['ထိုင်း']
-> ဘုရင် (score: 0.7317)
-> ဟံသာဝတီ (score: 0.7196)
-> ခေါ်တွင် (score: 0.7163)

--- FastText Analogy Test ---
Analogy: Positive ['နေပြည်တော်', 'မြန်မာ'] - Negative ['ထိုင်း']
-> အလံတော် (score: 0.8036)
-> တပ်မတော် (score: 0.8013)
-> ဆံတော် (score: 0.7922)

```

1.11 Step-by-Step Calculation ($King - Man + Woman = ?$)

$King - Man + Woman = ?$ ကိုပဲ အခြေခံပြီး ဘယ်လို တွက်ချက်တယ်ဆိုတာကို ရှင်းပြရရင်

1. ပထမဆုံး King ရဲ့ coordinate vector ကို ယူပါမယ်
2. ပြီးတဲ့အခါမှာ Man ရဲ့ coordinate vector ကိုလည်း ရှာဖွေပြီး (King - Man) နှုတ်တဲ့အပိုင်းကို လုပ်ပါတယ်။ ဆိုလိုတာက 'ဘုရင်' ဆိုတဲ့ စာလုံးကနေ 'ယောက်ျား' ဆိုတာကို နှုတ်လိုက်တာမို့လို့ မင်းမျိုးမင်းနွယ် ဆိုတဲ့ အဓိပ္ပါယ်လိုမျိုး ရသွားပါလိမ့်မယ်။
3. အဲဒီ တန်ဖိုးကိုမှ (+ Woman) လုပ်လိုက်တာမို့ 'မင်းမျိုးမင်းနွယ်' ဆိုတဲ့ အဓိပ္ပါယ်ကို 'အမျိုးသမီး' ထပ်ဖြည့်တဲ့သဘောပါ။
4. အဲဒီ vector ကို ကိုင်ပြီးတော့ nearest neighbor ရှာတွက်ကြည့်ရင် 'ဘုရင်မ' (Queen) ဆိုတဲ့ စာလုံးမျိုး ရလာတာပါ။ တွက်တဲ့အခါမှာတော့ Cosine similarity ကို သုံးပါတယ်။

```

[40]: def perform_analogy(model, positive_words, negative_words):
    print(f"Analogy: Positive {positive_words} - Negative {negative_words}")
    try:
        results = model.wv.most_similar(positive=positive_words,
    ↪negative=negative_words, topn=3)
        for word, sim in results:
            print(f"    -> {word} (score: {sim:.4f})")
    except KeyError as e:
        print(f"Skipped due to missing vocabulary item: {e}")

```

```
# Example analogy framework (ensure these terms exist in your big_corpus.txt
↳ vocabulary)
# Format: model.wv.most_similar(positive=['word_b', 'word_c'],
↳ negative=['word_a']) -> corresponds to word_a is to word_b as word_c is to ?
print("--- Word2Vec Analogy Test ---")
# Adjust these tokens based on common entities present in your corpus text
perform_analogy(w2v_model, positive_words=["စားခဲ့", "စား"], negative_words=["
↳ သွား"])

print("\n--- FastText Analogy Test ---")
perform_analogy(ft_model, positive_words=["စားခဲ့", "စား"], negative_words=["
↳ သွား"])
```

```
--- Word2Vec Analogy Test ---
Analogy: Positive ['စားခဲ့', 'စား'] - Negative ['သွား']
Skipped due to missing vocabulary item: "Key 'စားခဲ့' not present in vocabulary"
```

```
--- FastText Analogy Test ---
Analogy: Positive ['စားခဲ့', 'စား'] - Negative ['သွား']
-> အစားအစာ (score: 0.7842)
-> စားစရာ (score: 0.7774)
-> အစား (score: 0.7506)
```

အထက်မှာ မြင်ရတဲ့အတိုင်းပါပဲ မြန်မာစာအတွက်က အရမ်းအဆင်ပြေကြီး မဟုတ်ပါဘူး။ ဒေတာ များများ ထပ်ဖြည့်ရပါလိမ့်မယ်။ သို့သော်လည်း ကြိုးစားပြီး ရှာဖွေကြည့်တာ လုပ်ရင်တော့ ဥပမာကောင်းတွေ ရတတ်ပါတယ်။

1.12 Testing with English Language Pretrained Models

```
[25]: import os
import gensim.downloader as api
from gensim.models import KeyedVectors

# 1. Create local 'models' directory
models_dir = "./models"
os.makedirs(models_dir, exist_ok=True)

# 2. Download a lightweight model (GloVe 50-dimensional, ~65 MB)
model_path = os.path.join(models_dir, "glove_wiki_50.keyedvectors")

if not os.path.exists(model_path):
    print("Downloading lightweight GloVe model (~65 MB)...")
    # This downloads very quickly compared to the multi-gigabyte models
    model = api.load("glove-wiki-gigaword-50")
    model.save(model_path)
    print("Download complete and saved locally!")
else:
```

```
print("Loading local lightweight model...")
model = KeyedVectors.load(model_path)
```

Downloading lightweight GloVe model (~65 MB)...

```
[=====] 35.5% 23.4/66.0MB
```

downloaded

IOPub message rate exceeded.

The Jupyter server will temporarily stop sending output to the client in order to avoid crashing it.

To change this limit, set the config variable

`--ServerApp.iopub\_msg\_rate\_limit`.

Current values:

ServerApp.iopub_msg_rate_limit=1000.0 (msgs/sec)

ServerApp.rate_limit_window=3.0 (secs)

```
[=====] 100.0% 66.0/66.0MB
```

downloaded

Download complete and saved locally!

1.13 Let's Do Word Similarity Measurements and Vector Analogy Calculation Again

```
[41]: print("\n=====")
print("DEMONSTRATION 1: Word Similarity")
print("=====")
target_word = "computer"
print(f"Top 5 words similar to '{target_word}':")
for word, sim in model.most_similar(target_word, topn=5):
    print(f"  - {word}: {sim:.4f}")

print("\n=====")
print("DEMONSTRATION 2: Vector Analogy Calculation")
print("=====")
# King - Man + Woman = Queen
print("Running Analogy: king - man + woman")
for word, sim in model.most_similar(positive=['king', 'woman'],
    ↪negative=['man'], topn=3):
    print(f"  -> {word} (score: {sim:.4f})")
```

```
=====
DEMONSTRATION 1: Word Similarity
=====
```

Top 5 words similar to 'computer':

```
- computers: 0.9165
- software: 0.8815
```

- technology: 0.8526
- electronic: 0.8126
- internet: 0.8060

```
=====
DEMONSTRATION 2: Vector Analogy Calculation
=====
```

```
Running Analogy: king - man + woman
-> queen (score: 0.8524)
-> throne (score: 0.7664)
-> prince (score: 0.7592)
```

```
[48]: print("\n=====")
print("DEMONSTRATION 1: Word Similarity")
print("=====")
target_word = "superman"
print(f"Top 5 words similar to '{target_word}':")
for word, sim in model.most_similar(target_word, topn=5):
    print(f" - {word}: {sim:.4f}")

print("\n=====")
print("DEMONSTRATION 2: Vector Analogy Calculation")
print("=====")
# King - Man + Woman = Queen
print("Running Analogy: big - biggerr + small")
for word, sim in model.most_similar(positive=['big', 'small'],
    ↪negative=['bigger'], topn=5):
    print(f" -> {word} (score: {sim:.4f})")
```

```
=====
DEMONSTRATION 1: Word Similarity
=====
```

```
Top 5 words similar to 'superman':
- batman: 0.8642
- superboy: 0.8064
- superhero: 0.7787
- supergirl: 0.7405
- spider-man: 0.7293
```

```
=====
DEMONSTRATION 2: Vector Analogy Calculation
=====
```

```
Running Analogy: big - biggerr + small
-> large (score: 0.8236)
-> one (score: 0.8007)
-> along (score: 0.7915)
-> a (score: 0.7818)
```

```
-> known (score: 0.7778)
```

```
[52]: print("\n=====")
print("DEMONSTRATION 1: Word Similarity")
print("=====")
target_word = "researcher"
print(f"Top 5 words similar to '{target_word}':")
for word, sim in model.most_similar(target_word, topn=5):
    print(f"  - {word}: {sim:.4f}")

print("\n=====")
print("DEMONSTRATION 2: Vector Analogy Calculation")
print("=====")
# King - Man + Woman = Queen
print("Running Analogy: walk - walking + surf")
for word, sim in model.most_similar(positive=['walk', 'surf'],
    ↪negative=['walking'], topn=5):
    print(f"  -> {word} (score: {sim:.4f})")
```

```
=====
DEMONSTRATION 1: Word Similarity
=====
```

```
Top 5 words similar to 'researcher':
  - scientist: 0.8589
  - expert: 0.8174
  - professor: 0.8041
  - research: 0.8016
  - psychologist: 0.7962
```

```
=====
DEMONSTRATION 2: Vector Analogy Calculation
=====
```

```
Running Analogy: walk - walking + surf
  -> surfers (score: 0.7525)
  -> surfing (score: 0.6879)
  -> beach (score: 0.6575)
  -> swim (score: 0.6370)
  -> breakers (score: 0.6343)
```

1.14 Summary

ဒီ tutorial ကနေ Word2Vec နဲ့ FastText ကိုသုံးပြီး ဘယ်လို text embedding လုပ်သလဲ၊ ဒီ မော်ဒယ်နှစ်မျိုးကိုသုံးပြီး word similarity တွေကို ဘယ်လို တိုင်းတာလို့ ရသလဲ ဆိုတာနဲ့ ပတ်သက်ပြီး နားလည်သွားပြီလို့ ထင်ပါတယ်။ အသေးစိတ် သိဖို့က ကိုယ်တိုင် လက်တွေ့လုပ်ကြည့်ဖို့လည်း လိုအပ်သလို စာတမ်းတွေကို ဖတ်ကြည့်ကြဖို့လည်း အကြံပေးချင်ပါတယ်။

One more program for your study: <https://github.com/ye-kyaw-thu/tools/blob/master/>

`python/embedder.py`

[]: